

- 3 Leyer TDD

- Cod que mecano para que ese test pase

- Pasar del desarrollo condicido por testing (TDD)

- 1^{er} prueba → hardcoded data

VER CICLO ROSS, pg 9

10. Refactor

Refactoring → el cod tiene que hacer "exactamente" lo mismo
pero más limpio, más eficiente

↳ Por qué? → por cambios en el cod.

TDD sin refactoring "le falta una pta"

VER VIDEO

App p/ pedir taxis

Problemas TDD

• Testing sobre un escenario en particular

• Pro condiciones

• Necesitamos un usuario

- JSON p/ enviar y recibir

- Que el VERDE me retorne JSON con todos los datos

G3 - Comprar Entrada

- Técnica p/ desmontar software
- Testes vs TDD

Desmonte → Arma la prueba en
base a como funciona
Desmonta
→ Refactoring p/ volver a probar

ATDD "medio corre vamos a enter"

Orden del proceso - Probar unidades

Porque usar la técnica?

→ Cod robusto → fácil de cambiar
"Si donde cambiar"

- Concepto de "Refactor"

- Probar bien planteado

- Cod más segura

- Evitar duplicación de código

- Cobertura de pruebas

- Cómo voy a probar su API? → Cómo la he integrado en
APIs

Base de TDD → 1 Probar probar ROJO

2 Escribo cod p/ que prueba pase VERDE

3 Refactoring duplicado que hago? Términos de abstracción

→ Cambio cod → puede que fallan las pruebas

- Reporto & luego llamo un método volver al inicio(1)

que es solo la declaración → PASO
FALLA